

PAULT DOCUMENTATION

2026 02 22 Prompt Creation Launchpad

Source: docs/plans/2026-02-22-prompt-creation-launchpad.md

Theme: Clean Technical

Generated: February 26, 2026

Prompt Creation Launchpad — Implementation Plan

> **For Claude:** REQUIRED SUB-SKILL: Use superpowers:executing-plans to implement this plan task-by-task.

Goal: Replace the blank NewPromptView window with a Launchpad modal offering four starting paths (blank, template, AI generate, paste), a template system with bundled starters, and contextual coaching tips in the editor.

Architecture: New PromptTemplate SwiftData model + TemplateSeedService for bundled content. PromptLaunchpadView modal sheet replaces NewPromptView window. All paths funnel through existing PromptService.createPrompt() → PromptDetailView. Contextual coaching tips added to PromptDetailView empty state.

Tech Stack: SwiftUI, SwiftData, Swift Testing, macOS 15+

Task 1: PromptTemplate Model

Files:

- Create: Pault/PromptTemplate.swift
- Test: PaultTests/PromptTemplateTests.swift

Step 1: Write the failing test

Create PaultTests/PromptTemplateTests.swift:

```
//  
// PromptTemplateTests.swift  
// PaultTests  
//  
  
import Testing  
import SwiftData  
@testable import Pault  
  
@MainActor  
struct PromptTemplateTests {  
  
    private func makeContext() throws -> ModelContext {  
        let container = try ModelContainer(  
            for: PromptTemplate.self,  
            configurations: ModelConfiguration(isStoredInMemoryOnly: true)  
        )  
        return ModelContext(container)  
    }  
  
    @Test func templateCreatesWithDefaults() throws {  
        let ctx = try makeContext()  
        let template = PromptTemplate(  
            name: "Bug Report",  
            content: "## Bug\n{{description}}\n## Steps\n{{steps}}",  
            category: "Engineering"  
        )  
    }  
}
```

```
)
ctx.insert(template)
try ctx.save()

#expect(template.name == "Bug Report")
#expect(template.category == "Engineering")
#expect(template.isBuiltIn == false)
#expect(template.usageCount == 0)
#expect(template.iconName == "doc.text")
}

@Test func builtInTemplateCannotBeDeleted() throws {
    let template = PromptTemplate(
        name: "Starter",
        content: "Hello",
        category: "General",
        isBuiltIn: true
    )
    #expect(template.isBuiltIn == true)
}

@Test func usageCountIncrements() throws {
    let template = PromptTemplate(name: "T", content: "C", category: "X")
    #expect(template.usageCount == 0)
    template.usageCount += 1
    #expect(template.usageCount == 1)
}
}
```

Step 2: Run test to verify it fails

Run: `xcodebuild test -project Pault.xcodeproj -scheme Pault -testPlan PaultTests 2>&1 | tail -20`

Expected: FAIL — PromptTemplate type not found

Step 3: Write minimal implementation

Create Pault/PromptTemplate.swift:

```
//
// PromptTemplate.swift
// Pault
//

import Foundation
import SwiftData

@Model
final class PromptTemplate {
    var id: UUID
    var name: String
    var content: String
    var category: String
    var isBuiltIn: Bool
    var iconName: String
    var usageCount: Int
    var createdAt: Date
    var updatedAt: Date
}
```

```
init(
  name: String,
  content: String,
  category: String,
  isBuiltIn: Bool = false,
  iconName: String = "doc.text"
) {
  self.id = UUID()
  self.name = name
  self.content = content
  self.category = category
  self.isBuiltIn = isBuiltIn
  self.iconName = iconName
  self.usageCount = 0
  self.createdAt = Date()
  self.updatedAt = Date()
}
```

Step 4: Run test to verify it passes

Run: `xcodebuild test -project Pault.xcodeproj -scheme Pault -testPlan PaultTests`
`2>&1 | tail -20`

Expected: PASS — all 3 template tests pass

Step 5: Commit

```
git add Pault/PromptTemplate.swift PaultTests/PromptTemplateTests.swift
git commit -m "feat(templates): add PromptTemplate SwiftData model with tests"
```

Task 2: Register PromptTemplate in Schema

Files:

- Modify: `Pault/PaultApp.swift` (lines 43–52: schema array)

Step 1: Add PromptTemplate to schema

In `PaultApp.swift`, add `PromptTemplate.self` to the schema array at line 52 (before the closing `]`):

```
// Current (line 43-52):
let schema = Schema([
  Prompt.self,
  Tag.self,
  TemplateVariable.self,
  Attachment.self,
  PromptRun.self,
  CopyEvent.self,
  PromptVersion.self,
  SmartCollection.self,
])

// Change to:
let schema = Schema([
  Prompt.self,
  Tag.self,
```

```

TemplateVariable.self,
Attachment.self,
PromptRun.self,
CopyEvent.self,
PromptVersion.self,
SmartCollection.self,
PromptTemplate.self,
])

```

Step 2: Build to verify

Run: `xcodebuild build -project Pault.xcodeproj -scheme Pault 2>&1 | tail -5`

Expected: BUILD SUCCEEDED

Step 3: Run all tests

Run: `xcodebuild test -project Pault.xcodeproj -scheme Pault -testPlan PaultTests 2>&1 | tail -20`

Expected: All tests pass (existing + new template tests)

Step 4: Commit

```

git add Pault/PaultApp.swift
git commit -m "feat(templates): register PromptTemplate in SwiftData schema"

```

Task 3: TemplateSeedService

Files:

- Create: `Pault/Services/TemplateSeedService.swift`
- Test: `PaultTests/TemplateSeedServiceTests.swift`

Step 1: Write the failing test

Create `PaultTests/TemplateSeedServiceTests.swift`:

```

//
// TemplateSeedServiceTests.swift
// PaultTests
//

import Testing
import SwiftData
@testable import Pault

@MainActor
struct TemplateSeedServiceTests {

    private func makeContext() throws -> ModelContext {
        let container = try ModelContainer(
            for: PromptTemplate.self,
            configurations: ModelConfiguration(isStoredInMemoryOnly: true)
        )
        return ModelContext(container)
    }
}

```

```

@Test func seedCreatesBuiltInTemplates() throws {
    let ctx = try makeContext()
    TemplateSeedService.seed(into: ctx)

    let descriptor = FetchDescriptor<PromptTemplate>()
    let templates = try ctx.fetch(descriptor)

    #expect(templates.count >= 6)
    #expect(templates.allSatisfy(\.isBuiltIn))
    #expect(templates.contains(where: { $0.category == "Writing" }))
    #expect(templates.contains(where: { $0.category == "Engineering" }))
}

@Test func seedIsIdempotent() throws {
    let ctx = try makeContext()
    TemplateSeedService.seed(into: ctx)
    let countAfterFirst = try ctx.fetch(FetchDescriptor<PromptTemplate>()).count

    TemplateSeedService.seed(into: ctx)
    let countAfterSecond = try ctx.fetch(FetchDescriptor<PromptTemplate>()).count

    #expect(countAfterFirst == countAfterSecond)
}

@Test func seedTemplatesContainVariables() throws {
    let ctx = try makeContext()
    TemplateSeedService.seed(into: ctx)

    let templates = try ctx.fetch(FetchDescriptor<PromptTemplate>())
    let withVars = templates.filter { $0.content.contains("{") }
    #expect(withVars.count >= 3, "At least 3 bundled templates should use {{variables}}")
}

```

Step 2: Run test to verify it fails

Run: `xcodebuild test -project Pault.xcodeproj -scheme Pault -testPlan PaultTests 2>&1 | tail -20`

Expected: FAIL — TemplateSeedService not found

Step 3: Write implementation

Create `Pault/Services/TemplateSeedService.swift`:

```

//
// TemplateSeedService.swift
// Pault
//

import Foundation
import SwiftData
import os

private let seedLogger = Logger(subsystem: "com.pault.app", category: "TemplateSeed")

enum TemplateSeedService {

    /// Seeds built-in templates if none exist yet. Safe to call multiple times.

```

```

static func seed(into context: ModelContext) {
let descriptor = FetchDescriptor<PromptTemplate>(
predicate: #Predicate { $0.isBuiltIn == true }
)
let existingCount = (try? context.fetchCount(descriptor)) ?? 0
guard existingCount == 0 else {
seedLogger.debug("Built-in templates already seeded (\(existingCount) found)")
return
}

for def in bundledTemplates {
let template = PromptTemplate(
name: def.name,
content: def.content,
category: def.category,
isBuiltIn: true,
iconName: def.icon
)
context.insert(template)
}

do {
try context.save()
seedLogger.info("Seeded \(bundledTemplates.count) built-in templates")
} catch {
seedLogger.error("Failed to seed templates: \(error.localizedDescription)")
}
}

// MARK: - Bundled Template Definitions

private struct TemplateDef {
let name: String
let content: String
let category: String
let icon: String
}

private static let bundledTemplates: [TemplateDef] = [
TemplateDef(
name: "Email Drafter",
content: ""
Write a {{tone}} email to {{recipient}} about {{subject}}.

Key points to cover:
{{key_points}}

Keep it {{length}} and professional.
"",
category: "Writing",
icon: "envelope"
),
TemplateDef(
name: "Code Review Checklist",
content: ""
Review the following {{language}} code for:

1. Correctness – does it do what it claims?

```

2. Edge cases – what inputs could break it?
3. Performance – any obvious inefficiencies?
4. Readability – is it clear to a new developer?

Code to review:

```

{{code}}
"""
category: "Engineering",
icon: "checkmark.circle"
),
TemplateDef(
name: "Bug Report",
content: """
## Summary
{{summary}}

## Steps to Reproduce
1. {{step_1}}
2. {{step_2}}
3. {{step_3}}

## Expected Behavior
{{expected}}

## Actual Behavior
{{actual}}

## Environment
{{environment}}
"""
category: "Engineering",
icon: "ladybug"
),
TemplateDef(
name: "Meeting Notes Extractor",
content: """
Extract structured notes from the following meeting transcript:

{{transcript}}

Format as:
- **Decisions:** key decisions made
- **Action Items:** who does what by when
- **Open Questions:** unresolved topics
"""
category: "Productivity",
icon: "note.text"
),
TemplateDef(
name: "Content Summarizer",
content: """
Summarize the following {{content_type}} in {{length}} sentences:

{{content}}

Focus on the key takeaways and main arguments.
"""
category: "Writing",

```



```
icon: "text.justify.left"
),
TemplateDef(
name: "Creative Writing Starter",
content: ""
Write a {{genre}} story opening with:
- Setting: {{setting}}
- Main character: {{character}}
- Mood: {{mood}}

Start with an engaging hook that draws the reader in.
""",
category: "Writing",
icon: "pencil.and.outline"
),
TemplateDef(
name: "API Documentation",
content: ""
Document the following API endpoint:

**Endpoint:** {{method}} {{path}}
**Description:** {{description}}

### Request
{{request_body}}

### Response
{{response_body}}

### Error Codes
{{error_codes}}
""",
category: "Engineering",
icon: "doc.plaintext"
),
TemplateDef(
name: "Decision Framework",
content: ""
Help me decide between {{option_a}} and {{option_b}}.

Context: {{context}}

Evaluate each option on:
1. Cost/effort
2. Risk
3. Long-term impact
4. Reversibility

Recommend the better choice with reasoning.
""",
category: "Productivity",
icon: "arrow.triangle.branch"
),
]
}
```

Step 4: Run test to verify it passes

```
Run: xcodebuild test -project Pault.xcodeproj -scheme Pault -testPlan PaultTests
2>&1 | tail -20
```

Expected: PASS — all 3 seed tests pass

Step 5: Commit

```
git add Pault/Services/TemplateSeedService.swift
PaultTests/TemplateSeedServiceTests.swift
git commit -m "feat(templates): add TemplateSeedService with 8 bundled templates"
```

Task 4: Wire Seed into PaultApp

Files:

- Modify: Pault/PaultApp.swift (line 134: init())

Step 1: Call seed in init

In PaultApp.init() (line 134), add the seed call after the crash handler install and before the migration:

```
// Current init (lines 134-145):
init() {
    CrashReportingService.install()
    appDelegate.modelContainer = sharedModelContainer
    if UserDefaults.standard.string(forKey: "defaultAction") == "paste" {
        UserDefaults.standard.set("copy", forKey: "defaultAction")
    }
}

// Change to:
init() {
    CrashReportingService.install()
    appDelegate.modelContainer = sharedModelContainer

    // Seed built-in prompt templates on first launch
    let seedContext = ModelContext(sharedModelContainer)
    TemplateSeedService.seed(into: seedContext)

    if UserDefaults.standard.string(forKey: "defaultAction") == "paste" {
        UserDefaults.standard.set("copy", forKey: "defaultAction")
    }
}
```

Step 2: Build and run tests

```
Run: xcodebuild test -project Pault.xcodeproj -scheme Pault -testPlan PaultTests
2>&1 | tail -20
```

Expected: All tests pass

Step 3: Commit

```
git add Pault/PaultApp.swift
git commit -m "feat(templates): seed built-in templates on app launch"
```

Task 5: PromptService.createPromptFromTemplate

Files:

- Modify: `Pault/PromptService.swift` (add method after `createPrompt` at ~line 34)
- Test: `PaultTests/PromptServiceTests.swift` (add test)

Step 1: Write the failing test

Add to `PaultTests/PromptServiceTests.swift` (the `makeContext()` there already exists at line 14):

```
@Test func createPromptFromTemplatePopulatesContent() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    // Simulate a template (not using PromptTemplate model since it's not in this test container)
    let prompt = service.createPrompt(
        title: "Bug Report",
        content: "## Summary\n{{summary}}\n## Steps\n{{steps}}"
    )

    #expect(prompt.title == "Bug Report")
    #expect(prompt.content.contains("{{summary}}"))
    #expect(prompt.content.contains("{{steps}}"))
}
```

Note: The `createPromptFromTemplate` method is a convenience wrapper. Since `createPrompt(title:content:)` already exists, this test validates the flow. The actual `createPromptFromTemplate` method will increment usage count.

Step 2: Run test to verify it passes (this uses existing `createPrompt`)

Run: `xcodebuild test -project Pault.xcodeproj -scheme Pault -testPlan PaultTests 2>&1 | tail -20`

Expected: PASS

Step 3: Add `createPromptFromTemplate` to `PromptService`

In `Pault/PromptService.swift`, add after `createPrompt()` (after line 34):

```
/// Creates a new prompt pre-filled from a template and increments the template's usage count.
@discardableResult
func createPromptFromTemplate(_ template: PromptTemplate) -> Prompt {
    let prompt = createPrompt(title: template.name, content: template.content)
    template.usageCount += 1
    template.updatedAt = Date()
    save("createPromptFromTemplate")
    return prompt
}
```

Step 4: Build to verify

Run: `xcodebuild build -project Pault.xcodeproj -scheme Pault 2>&1 | tail -5`

Expected: BUILD SUCCEEDED

Step 5: Commit

```
git add Pault/PromptService.swift PaultTests/PromptServiceTests.swift
git commit -m "feat(templates): add createPromptFromTemplate to PromptService"
```

Task 6: PromptLaunchpadView — Core Shell

Files:

- Create: Pault/PromptLaunchpadView.swift

Step 1: Create the Launchpad view

Create Pault/PromptLaunchpadView.swift:

```
//
// PromptLaunchpadView.swift
// Pault
//

import SwiftUI
import SwiftData

#if os(macOS)
import AppKit
#endif

struct PromptLaunchpadView: View {
    @Environment(\.modelContext) private var modelContext
    @Environment(\.dismiss) private var dismiss

    @Query(
        filter: #Predicate<PromptTemplate> { $0.usageCount > 0 },
        sort: [SortDescriptor(\PromptTemplate.usageCount, order: .reverse)]
    ) private var recentTemplates: [PromptTemplate]

    @Query(sort: [SortDescriptor(\PromptTemplate.name, order: .forward)])
    private var allTemplates: [PromptTemplate]

    @State private var showTemplateBrowser = false
    @State private var showAIGenerator = false
    @State private var aiDescription = ""
    @State private var isGenerating = false
    @State private var searchText = ""

    private var service: PromptService { PromptService(modelContext: modelContext) }

    var body: some View {
        VStack(spacing: 0) {
            // Header
            HStack {
                Text("New Prompt")
                    .font(.title2)
                    .fontWeight(.semibold)
                Spacer()
                Button("Cancel") { dismiss() }
                    .keyboardShortcut(.cancelAction)
            }
            .padding()
        }
    }
}
```

```
Divider()

if showTemplateBrowser {
  templateBrowserView
} else if showAIGenerator {
  aiGeneratorView
} else {
  launchpadGrid
}
}
.frame(width: 520, height: 460)
}

// MARK: - Launchpad Grid

private var launchpadGrid: some View {
  VStack(spacing: 20) {
    Spacer()

    // Card grid – 2x2
    LazyVGrid(columns: [
      GridItem(.flexible(), spacing: 16),
      GridItem(.flexible(), spacing: 16)
    ], spacing: 16) {
      launchpadCard(
        icon: "doc.text",
        title: "Blank Prompt",
        subtitle: "Start from scratch",
        color: .blue
      ) {
        createBlankPrompt()
      }

      launchpadCard(
        icon: "rectangle.stack",
        title: "From Template",
        subtitle: "Use a template",
        color: .purple
      ) {
        showTemplateBrowser = true
      }

      if ProStatusManager.shared.isProUnlocked {
        launchpadCard(
          icon: "sparkles",
          title: "Generate with AI",
          subtitle: "Describe & generate",
          color: .orange,
          badge: "PRO"
        ) {
          showAIGenerator = true
        }
      } else {
        launchpadCard(
          icon: "sparkles",
          title: "Generate with AI",
          subtitle: "Describe & generate",
          color: .gray,
```

```
        badge: "PRO"
    ) {
        // Show paywall or nudge
        showAIGenerator = true
    }
}

launchpadCard(
    icon: "doc.on.clipboard",
    title: "Paste from Clipboard",
    subtitle: "Paste existing prompt",
    color: .green
) {
    createFromClipboard()
}
}
.padding(.horizontal, 32)

// Recent templates row
if !recentTemplates.isEmpty {
    VStack(alignment: .leading, spacing: 8) {
        Text("Recent Templates")
            .font(.subheadline)
            .foregroundStyle(.secondary)
            .padding(.horizontal, 32)

        ScrollView(.horizontal, showsIndicators: false) {
            HStack(spacing: 10) {
                ForEach(recentTemplates.prefix(5)) { template in
                    Button(action: { createFromTemplate(template) }) {
                        HStack(spacing: 6) {
                            Image(systemName: template.iconName)
                                .font(.caption)
                            Text(template.name)
                                .font(.caption)
                                .lineLimit(1)
                        }
                            .padding(.horizontal, 10)
                            .padding(.vertical, 6)
                            .background(Color.secondary.opacity(0.1))
                            .clipShape(Capsule())
                        }
                            .buttonStyle(.plain)
                    }
                }
            .padding(.horizontal, 32)
        }
    }

    Spacer()
}

// MARK: - Card Component

private func launchpadCard(
    icon: String,
```

```
title: String,
subtitle: String,
color: Color,
badge: String? = nil,
action: @escaping () -> Void
) -> some View {
    Button(action: action) {
        VStack(spacing: 10) {
            ZStack(alignment: .topTrailing) {
                Image(systemName: icon)
                    .font(.system(size: 28))
                    .foregroundColor(color)

                if let badge {
                    Text(badge)
                        .font(.system(size: 8, weight: .bold))
                        .padding(.horizontal, 4)
                        .padding(.vertical, 2)
                        .background(color.opacity(0.2))
                        .clipShape(Capsule())
                        .offset(x: 16, y: -8)
                }
            }

            Text(title)
                .font(.headline)
                .foregroundColor(.primary)

            Text(subtitle)
                .font(.caption)
                .foregroundColor(.secondary)
        }
        .frame(maxWidth: .infinity)
        .padding(.vertical, 20)
        .background(Color.secondary.opacity(0.06))
        .clipShape(RoundedRectangle(cornerRadius: 12))
        .overlay(
            RoundedRectangle(cornerRadius: 12)
                .stroke(Color.secondary.opacity(0.15), lineWidth: 1)
        )
        .buttonStyle(.plain)
    }

    // MARK: - Actions

    private func createBlankPrompt() {
        let prompt = service.createPrompt()
        postCreated(prompt)
        dismiss()
    }

    private func createFromClipboard() {
        #if os(macOS)
        let content = NSPasteboard.general.string(forType: .string) ?? ""
        #endif
        let prompt = service.createPrompt(content: content)
        TemplateEngine.syncVariables(for: prompt, in: modelContext)
    }
}
```

```
postCreated(prompt)
dismiss()
}

private fun createFromTemplate(_ template: PromptTemplate) {
    let prompt = service.createPromptFromTemplate(template)
    TemplateEngine.syncVariables(for: prompt, in: modelContext)
    postCreated(prompt)
    dismiss()
}

private fun postCreated(_ prompt: Prompt) {
    NotificationCenter.default.post(
        name: .promptCreated,
        object: nil,
        userInfo: ["promptID": prompt.id]
    )
}

// MARK: - Template Browser

private var templateBrowserView: some View {
    VStack(spacing: 0) {
        // Back button + search
        HStack {
            Button(action: { showTemplateBrowser = false }) {
                Label("Back", systemImage: "chevron.left")
            }
            .buttonStyle(.plain)

            Spacer()

            TextField("Search templates...", text: $searchText)
                .textFieldStyle(.roundedBorder)
                .frame(width: 200)
        }
        .padding(.horizontal)
        .padding(.vertical, 8)

        Divider()

        // Template list grouped by category
        ScrollView {
            let filtered = filteredTemplates
            let categories = Array(Set(filtered.map(\.category))).sorted()

            LazyVStack(alignment: .leading, spacing: 16) {
                ForEach(categories, id: \.self) { category in
                    Section {
                        ForEach(filtered.filter { $0.category == category }) { template in
                            Button(action: { createFromTemplate(template) }) {
                                HStack(spacing: 12) {
                                    Image(systemName: template.iconName)
                                        .font(.title3)
                                        .foregroundColor(.blue)
                                    .frame(width: 28)

                                }
                            }
                        }
                    }
                }
            }
        }
        .VStack(alignment: .leading, spacing: 2) {
```



```
Text(template.name)
    .font(.body)
    .fontWeight(.medium)
Text(template.content.prefix(80) + "...")
    .font(.caption)
    .foregroundStyle(.secondary)
    .lineLimit(1)
}

Spacer()

if template.isBuiltIn {
    Text("Built-in")
    .font(.caption2)
    .foregroundStyle(.secondary)
}
}
.padding(.vertical, 6)
.padding(.horizontal, 12)
.contentShape(Rectangle())
}
.buttonStyle(.plain)
}
} header: {
    Text(category)
    .font(.subheadline)
    .fontWeight(.semibold)
    .foregroundStyle(.secondary)
    .padding(.horizontal, 12)
}
}
}
.padding()
}
}
}

private var filteredTemplates: [PromptTemplate] {
    if searchText.isEmpty { return Array(allTemplates) }
    let query = searchText.lowercased()
    return allTemplates.filter {
        $0.name.lowercased().contains(query) ||
        $0.category.lowercased().contains(query) ||
        $0.content.lowercased().contains(query)
    }
}

// MARK: - AI Generator

private var aiGeneratorView: some View {
    VStack(spacing: 16) {
        // Back button
        HStack {
            Button(action: { showAIGenerator = false }) {
                Label("Back", systemImage: "chevron.left")
            }
            .buttonStyle(.plain)
        }
        Spacer()
    }
}
```

```
}  
.padding(.horizontal)  
.padding(.top, 8)  
  
Spacer()  
  
if !ProStatusManager.shared.isProUnlocked {  
    // Pro nudge  
    VStack(spacing: 12) {  
        Image(systemName: "sparkles")  
        .font(.system(size: 36))  
        .foregroundColor(.orange)  
        Text("AI Generation is a Pro feature")  
        .font(.headline)  
        Text("Upgrade to Pro to generate prompts from descriptions.")  
        .font(.subheadline)  
        .foregroundColor(.secondary)  
        .multilineTextAlignment(.center)  
    }  
} else {  
    VStack(spacing: 12) {  
        Image(systemName: "sparkles")  
        .font(.system(size: 36))  
        .foregroundColor(.orange)  
        Text("Describe your prompt")  
        .font(.headline)  
        Text("Tell us what you want your prompt to do, and AI will generate it.")  
        .font(.subheadline)  
        .foregroundColor(.secondary)  
        .multilineTextAlignment(.center)  
  
        TextEditor(text: $aiDescription)  
        .font(.body)  
        .frame(height: 100)  
        .scrollContentBackground(.hidden)  
        .padding(8)  
        .background(Color.secondary.opacity(0.06))  
        .clipShape(RoundedRectangle(cornerRadius: 8))  
        .overlay(  
            RoundedRectangle(cornerRadius: 8)  
            .stroke(Color.secondary.opacity(0.2), lineWidth: 1)  
        )  
        .padding(.horizontal, 32)  
  
        Button(action: generateFromAI) {  
            if isGenerating {  
                ProgressView()  
                .controlSize(.small)  
            } else {  
                Label("Generate", systemImage: "sparkles")  
            }  
        }  
        .buttonStyle(.borderedProminent)  
        .disabled(aiDescription.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty ||  
            isGenerating)  
    }  
}
```

```

Spacer()
}
}

private func generateFromAI() {
guard !aiDescription.isEmpty else { return }
isGenerating = true

let description = aiDescription
let config = AIConfig.defaults[.claude] ?? AIConfig(provider: .claude, model:
"claude-opus-4-6")

Task {
do {
let content = try await AIService.shared.generatePrompt(
from: description,
config: config
)
await MainActor.run {
let prompt = service.createPrompt(content: content)
TemplateEngine.syncVariables(for: prompt, in: modelContext)
postCreated(prompt)
isGenerating = false
dismiss()
}
} catch {
await MainActor.run {
isGenerating = false
// Fall back: create prompt with the description as content
let prompt = service.createPrompt(content: description)
postCreated(prompt)
dismiss()
}
}
}
}
}
}
}

```

Step 2: Build to verify

Run: `xcodebuild build -project Pault.xcodeproj -scheme Pault 2>&1 | tail -10`

Expected: May fail because `AIService.generatePrompt` doesn't exist yet. That's fine — we'll add it in Task 7.

Step 3: Commit

```

git add Pault/PromptLaunchpadView.swift
git commit -m "feat(launchpad): add PromptLaunchpadView with card grid, template
browser, and AI generator"

```

Task 7: `AIService.generatePrompt`

Files:

- Modify: `Pault/Services/AIService.swift` (add method after `improve()` at ~line 83)

Step 1: Add `generatePrompt` method

In `AIService.swift`, add after the `improve()` method (after line 83):

```
func generatePrompt(from description: String, config: AIConfig) async throws -> String {
    let system = ""
    You are an expert prompt engineer. Based on the user's description, \
    create a well-structured, reusable prompt template. \
    Use {{variable_name}} syntax for parts the user should fill in each time. \
    Return ONLY the prompt text, no commentary or explanation.
    ""
    return try await complete(system: system, user: description, config: config)
}
```

Step 2: Build to verify

Run: `xcodebuild build -project Pault.xcodeproj -scheme Pault 2>&1 | tail -5`

Expected: BUILD SUCCEEDED (PromptLaunchpadView can now resolve `AIService.shared.generatePrompt`)

Step 3: Run all tests

Run: `xcodebuild test -project Pault.xcodeproj -scheme Pault -testPlan PaultTests 2>&1 | tail -20`

Expected: All tests pass

Step 4: Commit

```
git add Pault/Services/AIService.swift
git commit -m "feat(ai): add generatePrompt method to AIService for launchpad AI generation"
```

Task 8: Wire Launchpad into ContentView + PaultApp

Files:

- Modify: `Pault/ContentView.swift` (lines 72-75: new prompt button, lines 128-130: notification handler)
- Modify: `Pault/PaultApp.swift` (lines 117-122: remove `Window("New Prompt")`)

Step 1: Update ContentView

Add state variable (after line 28, with other `@State` declarations):

```
@State private var showCreationLaunchpad: Bool = false
```

Replace the `openWindow(id: "new-prompt")` button (line 72-75):

```
// Old:
Button(action: { openWindow(id: "new-prompt") }) {
    Image(systemName: "plus")
}
.help("New Prompt (■N)")

// New:
Button(action: { showCreationLaunchpad = true }) {
    Image(systemName: "plus")
}
.help("New Prompt (■N)")
```

Replace the `.onReceive` for `createNewPrompt` (line 128-130):

```
// Old:
.receive(NotificationCenter.default.publisher(for: .createNewPrompt)) { _ in
  openWindow(id: "new-prompt")
}

// New:
.receive(NotificationCenter.default.publisher(for: .createNewPrompt)) { _ in
  showCreationLaunchpad = true
}
```

Add the sheet modifier (after the `.sheet(isPresented: $showingAnalytics)` block at ~line 127):

```
.sheet(isPresented: $showCreationLaunchpad) {
  PromptLaunchpadView()
}
```

Step 2: Remove the Window("New Prompt") from PaultApp

In `PaultApp.swift`, delete lines 117-122:

```
// DELETE these lines:
Window("New Prompt", id: "new-prompt") {
  NewPromptView()
}
.windowResizability(.contentMinSize)
.defaultSize(AppConstants.Windows.promptDefault)
.modelContainer(sharedModelContainer)
```

Step 3: Build and run tests

Run: `xcodebuild test -project Pault.xcodeproj -scheme Pault -testPlan PaultTests 2>&1 | tail -20`

Expected: All tests pass. Note: `NewPromptView.swift` can remain in the project — it's unused but doesn't hurt. Consider removing later.

Step 4: Commit

```
git add Pault/ContentView.swift Pault/PaultApp.swift
git commit -m "feat(launchpad): wire PromptLaunchpadView as modal sheet, remove separate
creation window"
```

Task 9: Contextual Coaching Tips in PromptDetailView

Files:

- Modify: `Pault/PromptDetailView.swift` (add coaching view between title field and content editor)

Step 1: Add AppStorage flags and coaching view

Add `@AppStorage` flags (after line 38, with other state declarations):

```
@AppStorage("coachingDismissedVariables") private var coachingDismissedVariables = false
@AppStorage("coachingDismissedTags") private var coachingDismissedTags = false
@AppStorage("hasDiscoveredAIAssist") private var hasDiscoveredAIAssist = false
```

Add a computed property for the current coaching tip (after the state declarations):

```
private var coachingTip: (message: String, icon: String)? {
  if prompt.content.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty
  && !coachingDismissedVariables {
```

```

return ("Use {{variable_name}} to create reusable placeholders", "lightbulb")
}
if !prompt.content.isEmpty
&& prompt.templateVariables.isEmpty
&& !coachingDismissedVariables {
return ("Add {{variables}} to make this prompt reusable across different contexts",
"lightbulb")
}
if !prompt.templateVariables.isEmpty
&& prompt.tags.isEmpty
&& !coachingDismissedTags {
return ("Add tags to organize and find your prompts quickly", "tag")
}
return nil
}

```

Insert the coaching tip view in the body, between the title field (line 51) and the content editor (line 57). Add after the title `.padding(.bottom, 12)`:

```

// Contextual coaching tip
if let tip = coachingTip {
HStack(spacing: 8) {
Image(systemName: tip.icon)
.foregroundColor(.blue)
.font(.caption)
Text(tip.message)
.font(.caption)
.foregroundColor(.secondary)
Spacer()
Button(action: dismissCurrentTip) {
Image(systemName: "xmark")
.font(.caption2)
.foregroundColor(.tertiary)
}
.buttonStyle(.plain)
}
.padding(.horizontal, 20)
.padding(.vertical, 8)
.background(Color.blue.opacity(0.05))
.transition(.move(edge: .top).combined(with: .opacity))
}

```

Add the dismiss method (near the other private methods):

```

private func dismissCurrentTip() {
withAnimation {
if prompt.content.isEmpty || prompt.templateVariables.isEmpty {
coachingDismissedVariables = true
} else if prompt.tags.isEmpty {
coachingDismissedTags = true
}
}
}
}

```

Step 2: Add blue dot on AI Assist button

Modify the AI Assist button (around line 142-152) to show a discovery badge:

```
// Current AI Assist button:
Button(action: {
guard ProStatusManager.shared.isProUnlocked else { showPaywall = true; return }
showAIPanel.toggle()
}) {
Image(systemName: "sparkles")
.font(.title2)
.foregroundColor(showAIPanel ? .blue : .secondary)
.padding(12)
}

// Change to:
Button(action: {
guard ProStatusManager.shared.isProUnlocked else { showPaywall = true; return }
showAIPanel.toggle()
if !hasDiscoveredAIAssist { hasDiscoveredAIAssist = true }
}) {
ZStack(alignment: .topTrailing) {
Image(systemName: "sparkles")
.font(.title2)
.foregroundColor(showAIPanel ? .blue : .secondary)
.padding(12)

if ProStatusManager.shared.isProUnlocked && !hasDiscoveredAIAssist {
Circle()
.fill(.blue)
.frame(width: 8, height: 8)
.offset(x: -4, y: 4)
}
}
}
}
```

Step 3: Build and run tests

Run: `xcodebuild test -project Pault.xcodeproj -scheme Pault -testPlan PaultTests 2>&1 | tail -20`

Expected: All tests pass

Step 4: Commit

```
git add Pault/PromptDetailView.swift
git commit -m "feat(coaching): add contextual tips and AI discovery badge to
PromptDetailView"
```

Task 10: Save as Template

Files:

- Modify: `Pault/PromptDetailView.swift` (add save-as-template action)

Step 1: Add state for save-as-template sheet

Add to state declarations in `PromptDetailView`:

```
@State private var showSaveAsTemplate = false
@State private var templateName = ""
@State private var templateCategory = "General"
```

Step 2: Add "Save as Template" to the toolbar/action area

Add a context menu or additional button near the existing toolbar buttons. The cleanest approach is adding to the overlay area. Add after the inspector toggle button (around line 177):

```
// Save as Template button
Button(action: {
    templateName = prompt.title
    showSaveAsTemplate = true
}) {
    Image(systemName: "rectangle.stack.badge.plus")
    .font(.title2)
    .foregroundColor(.secondary)
    .padding(12)
}
.buttonStyle(.plain)
.help("Save as Template")
```

Step 3: Add the sheet

Add the sheet modifier (after the existing .sheet modifiers):

```
.sheet(isPresented: $showSaveAsTemplate) {
    SaveAsTemplateSheet(
        name: $templateName,
        category: $templateCategory,
        onSave: {
            let template = PromptTemplate(
                name: templateName,
                content: prompt.content,
                category: templateCategory
            )
            modelContext.insert(template)
            try? modelContext.save()
            showSaveAsTemplate = false
        },
        onCancel: { showSaveAsTemplate = false }
    )
}
```

Step 4: Create the SaveAsTemplateSheet view

Add at the bottom of PromptDetailView.swift (or as a separate section):

```
private struct SaveAsTemplateSheet: View {
    @Binding var name: String
    @Binding var category: String
    let onSave: () -> Void
    let onCancel: () -> Void

    private let categories = ["General", "Writing", "Engineering", "Productivity",
        "Analysis"]

    var body: some View {
        VStack(spacing: 0) {
            HStack {
                Text("Save as Template")
                .font(.headline)
                Spacer()
                Button("Cancel", action: onCancel)
            }
        }
    }
}
```



```

.keyboardShortcut(.cancelAction)
}
.padding()

Divider()

Form {
  TextField("Template Name", text: $name)
  Picker("Category", selection: $category) {
    ForEach(categories, id: \.self) { cat in
      Text(cat).tag(cat)
    }
  }
}
.formStyle(.grouped)

Divider()

HStack {
  Spacer()
  Button("Save Template", action: onSave)
    .buttonStyle(.borderedProminent)
    .disabled(name.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty)
}
.padding()
}
.frame(width: 360, height: 240)
}
}

```

Step 5: Build and run tests

Run: `xcodebuild test -project Pault.xcodeproj -scheme Pault -testPlan PaultTests 2>&1 | tail -20`

Expected: All tests pass

Step 6: Commit

```

git add Pault/PromptDetailView.swift
git commit -m "feat(templates): add Save as Template action to PromptDetailView"

```

Task 11: Final Integration Test + Cleanup

Files:

- All modified files
- Optional: remove or deprecate `Pault/NewPromptView.swift`

Step 1: Full build

Run: `xcodebuild build -project Pault.xcodeproj -scheme Pault 2>&1 | tail -10`

Expected: BUILD SUCCEEDED

Step 2: Run full test suite

```
Run: xcodebuild test -project Pault.xcodeproj -scheme Pault -testPlan PaultTests
2>&1 | tail -30
```

Expected: All tests pass

Step 3: Verify no compiler warnings

```
Run: xcodebuild build -project Pault.xcodeproj -scheme Pault 2>&1 | grep -i
warning | head -10
```

Expected: No new warnings (or only pre-existing ones)

Step 4: Optional — Add deprecation note to NewPromptView

If NewPromptView is still referenced anywhere (e.g., the Preview), leave it. Otherwise add a comment at the top:

```
// DEPRECATED: Replaced by PromptLaunchpadView in v3.0. Kept for reference.
```

Step 5: Final commit

```
git add -A
git commit -m "chore: final integration verification for prompt creation launchpad"
```

Verification

Unit Tests

- PromptTemplate model creation and properties (Task 1)
- TemplateSeedService idempotency and template content (Task 3)
- createPromptFromTemplate() usage count increment (Task 5)

Manual Testing

1. Cmd+N → Launchpad modal appears (not a separate window)
2. "Blank Prompt" card → empty PromptDetailView
3. "From Template" → browse → select → pre-filled PromptDetailView
4. "Generate with AI" (Pro) → describe → generated content in PromptDetailView
5. "Paste from Clipboard" → clipboard content in PromptDetailView
6. Return creates blank prompt immediately
7. Recent Templates row shows most-used (after using a template once)
8. Coaching tips appear on blank editor, dismiss on "x"
9. Blue dot on AI Assist disappears after first open
10. "Save as Template" → new template appears in template browser
11. Bundled templates visible in "From Template" grouped by category
12. Template search filters correctly